UNIVERSIDAD NACIONAL DEL SANTA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA

DE SISTEMAS E INFORMÁTICA

"S03_04 - Arquitectura BDI y Aprendizaje por Refuerzo"

Integrantes:

Rodriguez Melendez Jeffri

Aquino Reyna Julio

Asignatura: Agentes Inteligentes

Ciclo: VIII

Semestre: 2026 – II

Docente: Borja Reyna Whiston

NUEVO CHIMBOTE – PERÚ

2026

1. Introducción

Este informe documenta la práctica calificada de laboratorio de las semanas 03 y 04 del curso de Agentes Inteligentes. La práctica tiene dos partes independientes. En la primera se programa un agente deliberativo con arquitectura BDI (Creencias, Deseos, Intenciones) que supervisa un clúster de microservicios. En la segunda se implementa Q-Learning en un entorno estocástico tipo FrozenLake y se mide si la ingeniería de recompensas (reward shaping) acelera la convergencia.

Todo lo reportado aquí sale de ejecutar los dos archivos de la solución (ejercicio1.py y ejercicio_2_reward_shaping_vs_estandar.py). Las cifras del Ejercicio 2 se complementan con una repetición del experimento sobre 30 semillas distintas, para no depender de una sola corrida afortunada.

2. Ejercicio 1: Agente BDI con inferencia deductiva y reconsideración dinámica

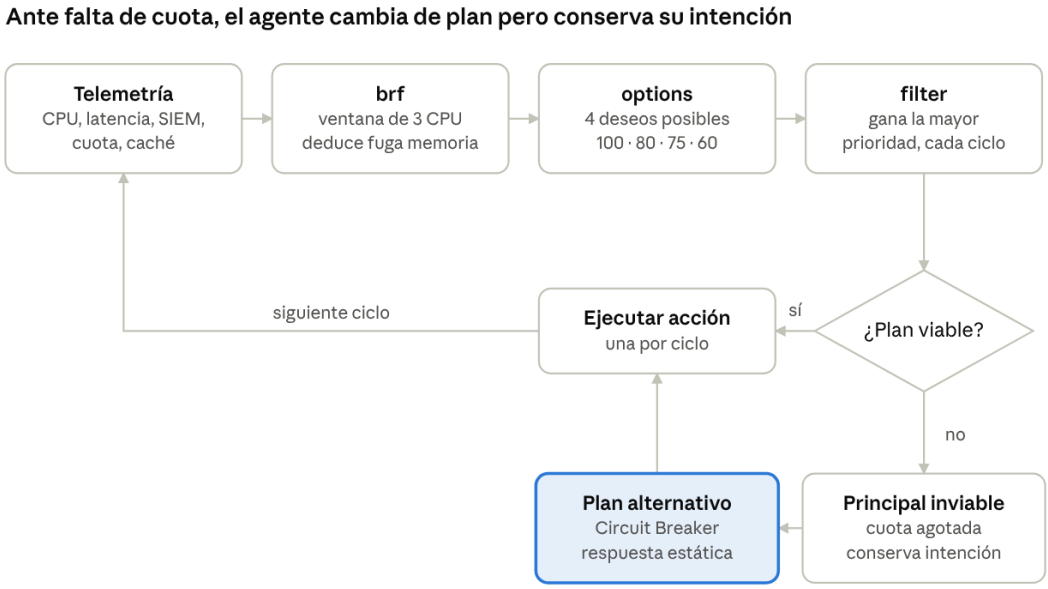
2.1 Diseño del agente

El agente sigue el intérprete de Wooldridge. En cada ciclo recibe una telemetría del clúster (CPU, latencia, alerta del SIEM, cuota de nube y estado de la caché), revisa sus creencias, genera deseos, filtra uno como intención y ejecuta una acción del plan elegido. Como la intención se reconsidera en cada ciclo, el agente tiene un compromiso de mente abierta: puede abandonar lo que hacía si aparece algo más urgente.

La regla de inferencia deductiva vive en brf. Se guarda una ventana deslizante con las últimas 3 lecturas de CPU, y si son estrictamente crecientes y cada intervalo sube más de 15 puntos porcentuales, el agente deduce anomalia_fuga_memoria = True, aunque la CPU todavía no llegue al 80 %. La creencia se recalcula en cada ciclo, así que no queda activada para siempre.

Las prioridades son las de la guía: seguridad 100, caché 80, OOM 75 y SLA 60. filter elige siempre el deseo de mayor prioridad entre los activos.

Para el SLA hay dos planes en la biblioteca. El principal (Runbook_Autoescalado_Horizontal) solo es aplicable si la cuota de nube no está agotada. El alternativo (Runbook_Degradacion_Elegante_CircuitBreaker) no tiene restricciones. Cuando la cuota se agota a mitad de plan, el agente marca el principal como fallido, vacía la cola de acciones, conserva la intención y selecciona el alternativo. El marcador de plan fallido solo se borra cuando cambia la intención, y eso evita que el agente reintente el plan fallido en bucle.



ciclo del agente BDI · 5 etapas, 1 decisión

El agente repite este ciclo con cada telemetría; si el plan principal deja de ser viable, la intención no se toca y solo cambia el plan.

2.2 Decisiones de diseño propias

La guía deja algunos puntos abiertos y la solución los resolvió así:

La guía pide una tasa de incremento superior a +15 % por ciclo, pero el comentario de la plantilla sugería `p3 - p1 >= 15`. Se eligió la lectura más estricta y fiel al enunciado: cada intervalo debe superar 15 puntos porcentuales. Con ciclos consecutivos y equidistantes, la regla del comentario dispararía alertas con crecimientos mucho más suaves.

La plantilla no trae ninguna señal para detectar una caída de caché, así que se agregó el campo explícito `cache_caida` en la telemetría, en lugar de deducirlo de la latencia. Del mismo modo, la guía no fija el umbral de latencia del SLA, y se adoptó 200 ms. El deseo de SLA se activa si la CPU llega al 80 % o la latencia supera ese umbral. Los planes de seguridad, caché y OOM no están especificados en la guía; son propuestas razonables para la simulación.

2.3 Resultados de la simulación

Se ejecutaron tres escenarios.

Escenario 1: inferencia antes del 80 %. Con lecturas de CPU de 40, 58 y 75 %, el agente no hace nada en los ciclos 1 y 2. En el ciclo 3 la ventana [40, 58, 75] tiene incrementos de 18 y 17 puntos, por lo que se deduce la fuga de memoria con la CPU todavía en 75 %. Se activa PREVENIR_CAIDA_OOM, se selecciona `Runbook_Prevencion_OOM` y se ejecuta CAPTURAR_DIAGNOSTICO_MEMORIA. Esto cumple el requisito de alerta previa al umbral crítico.

Escenario 2: reconsideración ante un ataque. En el ciclo 4 llegan CPU 92 %, latencia 300 ms, alerta del SIEM y caché caída. La ventana [58, 75, 92] sigue disparando OOM, así que compiten los cuatro deseos a la vez. Gana seguridad (prioridad 100), el agente abandona la intención OOM y pasa a Runbook_Seguridad con BLOQUEAR_ORIGEN_MALICIOSO.

Escenario 3: cuota agotada durante el plan principal. En el ciclo 1 el agente elige Runbook_Autoescalado_Horizontal y ejecuta APROVISIONAR_NODO_REPLICA_K8S. En el ciclo 2 la cuota se agota: el plan principal se declara inviable, la intención ESTABILIZAR_RENDIMIENTO_SLA se mantiene y el agente conmuta al plan de Circuit Breaker, ejecutando ACTIVAR_CIRCUIT_BREAKER_RECOMENDACIONES. En el ciclo 3 completa HABILITAR_RESPUESTA_ESTATICA_CDN y termina el plan. En el ciclo 4 la telemetría vuelve a la normalidad (CPU 50 %, latencia 100 ms), no quedan deseos activos y la intención se libera.

2.4 Cumplimiento de la rúbrica

La inferencia deductiva con ventana de 3 lecturas activa la alerta antes del 80 % (Escenario 1). La priorización funciona y el cambio de intención por reconsideración está demostrado (Escenario 2). El fallback conmuta al plan alternativo preservando la intención y sin reintentos infinitos del plan fallido (Escenario 3).

3. Ejercicio 2: Q-Learning en entorno estocástico y reward shaping

3.1 Entorno y algoritmo

El entorno es un gridworld de 4×4 con 16 estados, 4 acciones, hoyos en los estados 5, 7, 11 y 12, y meta en el estado 15. Es estocástico: la acción deseada se ejecuta con probabilidad 1/3 y con probabilidad 1/3 cada una se desvía a una de las dos direcciones laterales, igual que FrozenLake con is_slippery=True.

Se implementó la actualización de Bellman sin modelo tal como pide la guía: TD-Target = r + γ·max Q(s', a'), TD-Error = TD-Target − Q(s, a) y Q(s, a) ← Q(s, a) + α·TD-Error. La exploración es ε-greedy con decaimiento exponencial de $\varepsilon_0 = 1.0$ hasta $\varepsilon_{min} = 0.01$ con factor 0.995 por episodio. Los hiperparámetros usados son $\alpha = 0.2$, $\gamma = 0.95$, 2000 episodios y un máximo de 100 pasos por episodio.

Hay dos esquemas de recompensa. En el estándar solo hay +1.0 al llegar a la meta y 0.0 en todo lo demás. Con reward shaping hay −0.01 por cada paso normal, −1.0 al caer en un hoyo y +1.0 en la meta.

3.2 Correcciones respecto a la plantilla

La solución se aparta de la plantilla en tres puntos, y conviene que el informe lo diga porque son decisiones técnicas defendibles:

En los estados terminales el TD-Target es solo la recompensa, sin sumar $\gamma \cdot \max Q$ del estado siguiente. La plantilla hacía bootstrap desde estados terminales, lo cual es incorrecto porque desde un hoyo o la meta no hay futuro. La selección greedy desempata al azar entre acciones con el mismo valor Q, porque con la tabla inicializada en ceros argmax siempre devolvería la acción 0 y sesgaría la exploración. Y se agregó un límite de pasos por episodio y una semilla reproducible. Además, α bajó de 0.8 a 0.2, porque con 0.8 las actualizaciones son demasiado agresivas para un entorno tan ruidoso.

3.3 Resultados: corrida con semilla 10

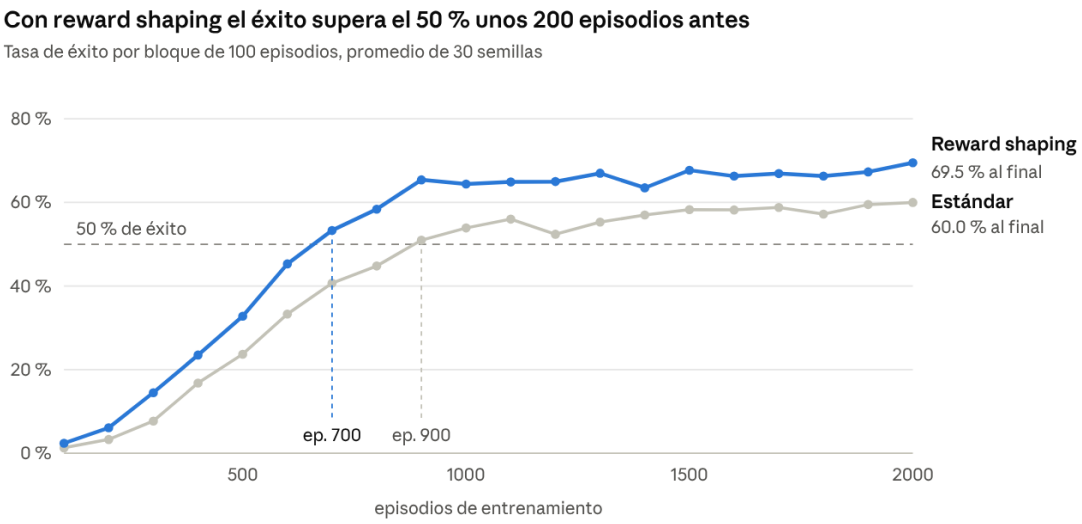
La tasa de éxito se midió en bloques de 100 episodios durante el entrenamiento. En la corrida con semilla 10, el esquema estándar arranca en 2 % y llega a 58 % hacia el episodio 900, pero después oscila entre 41 % y 63 %. Con reward shaping arranca en 6 %, cruza el 48 % en el bloque 501–600 y se estabiliza entre 56 % y 72 %.

Los valores Q del estado inicial al final del entrenamiento fueron, para el esquema estándar, [0.116, 0.111, 0.132, 0.106] y para reward shaping, [−0.078, −0.111, −0.111, −0.110] (acciones: izquierda, abajo, derecha, arriba). Los valores negativos del segundo caso no son un error: reflejan el costo acumulado por paso y el riesgo de hoyo. Lo que importa es la diferencia entre acciones. En el esquema estándar los cuatro valores son casi iguales y la política apenas distingue entre acciones. Con shaping la acción izquierda se separa claramente de las demás, y es coherente con lo conocido de FrozenLake resbaladizo: desde la esquina inicial, empujar contra la pared es la opción más segura porque ningún desvío puede mandar al agente a un hoyo.

3.4 Resultados: promedio de 30 semillas

Una sola semilla puede engañar, así que el experimento se repitió con las semillas 0 a 29 en ambos esquemas, con la misma configuración.

En los últimos 500 episodios la tasa de éxito media es de 58.7 % con el esquema estándar y de 67.2 % con reward shaping. La variabilidad entre semillas también es menor con shaping (desviación de 2.3 puntos frente a 4.1). En la velocidad de convergencia, el promedio de episodios necesarios para que el bloque de 100 supere el 50 % de éxito fue de unos 907 con el esquema estándar y unos 717 con shaping, es decir, unos 190 episodios menos. Shaping llegó antes en 23 de las 30 semillas, empató en 6 y perdió solo en 1.



ejercicio_2_reward_shaping_vs_estandar.py · semillas 0–29, 2000 episodios

Cada punto es el promedio de 30 corridas independientes; las líneas punteadas verticales marcan el primer bloque que supera el 50 % en cada esquema.

3.5 Análisis

Los datos respaldan la hipótesis de la guía: la penalización por paso y por hoyo acelera la convergencia y además mejora la tasa de éxito final. La explicación es de señal de aprendizaje. Con recompensa dispersa, el agente no recibe ninguna información hasta que alcanza la meta por azar, algo raro en este entorno resbaladizo, así que durante cientos de episodios casi todos los valores Q siguen en cero. Con shaping, cada paso y cada caída producen información inmediata, y los valores Q se diferencian mucho antes.

Hay límites que deben decirse con claridad. La guía pide demostrar que el shaping evita trayectorias en bucle; este experimento no midió la longitud de las trayectorias, así que eso se infiere del diseño de la penalización por paso (cada paso extra cuesta) pero no está medido directamente. La tasa de éxito se mide durante el entrenamiento, con ε en su mínimo de 0.01, no con una evaluación separada de la política final. Y el techo de éxito cercano a 67 % se debe a la estocasticidad del entorno: aun con una política muy buena, el viento hace que muchos episodios terminen en un hoyo. Por último, el shaping puede cambiar la política óptima si las penalizaciones son grandes; con -0.01 por paso es suficientemente pequeño para no desviar al agente de la meta.

4. Conclusiones

En el Ejercicio 1, el agente BDI cumple los tres comportamientos pedidos: deduce la fuga de memoria antes de que la CPU llegue al 80 %, reconsidera su intención cuando aparece una prioridad mayor y conmuta al plan alternativo ante la falta de cuota sin perder la intención ni caer en reintentos infinitos. La pieza que lo hace posible es separar la intención (qué se quiere lograr) del plan (cómo se logra): el fallo de un plan no destruye la meta.

En el Ejercicio 2, Q-Learning con reward shaping convergió antes y con menos variabilidad que con recompensa dispersa en 23 de 30 semillas, con una mejora de casi 9 puntos porcentuales en la tasa de éxito final. Lo más útil que deja la práctica es que el diseño de la función de recompensa es una decisión de ingeniería tan importante como el algoritmo, y que conviene validarla con varias semillas antes de sacar conclusiones.

5. Archivos de la solución

practica-03_04/ejercicio1.py contiene el agente BDI completo con los tres escenarios de prueba. practica-03_04/ejercicio2.py contiene el entorno, el algoritmo Q-Learning y el experimento comparativo entre ambos esquemas de recompensa.